



UNIVERSIDADE FEDERAL DE ITAJUBÁ

Analizador Léxico e Sintático da Linguagem
Project Hail Mary (PHM++)

1º Trabalho ECOM06A – Compiladores

Pedro Henrique Corsini Soares Rodrigues – 2024004107

Pedro Henrique Moreira de Mello Balbino – 2024003988

Itajubá
Maio de 2026

Sumário

1	Introdução	2
2	Desenvolvimento	3
2.1	Regras da Linguagem	3
2.2	Análise Léxica	3
2.3	Análise Sintática	7
2.4	Autômatos Finitos	8
3	Exemplo Funcional da Linguagem PHM++	15
3.1	Código-Fonte do Programa	15
3.2	Mapeamento Léxico e Sintático	17
3.3	Observações sobre o Exemplo	18
4	Conclusão	19
	Referências	20

1 Introdução

Os compiladores são um componente de extrema importância para a computação atual. Com eles, o desenvolvimento de softwares não se restringe à manipulação de hardware de baixo nível, tornando possível a existência de abstrações de alto nível. Dessa forma, para construir uma linguagem de programação, os analisadores léxico e sintático são etapas essenciais em um compilador, garantindo a produção de um código coerente e que a máquina consiga lê-lo e executar suas instruções com o funcionamento esperado e determinado pelo usuário que a criou.

O analisador léxico tem como principal função a leitura dos caracteres do código-fonte, agrupando-os em *lexemas* e gerando um fluxo de *tokens* de saída. Um lexema é a sequência de caracteres no código-fonte que corresponde ao padrão de um *token*, sendo identificado pelo analisador léxico como uma instância específica dessa categoria. Um *token* é um par que consiste de um nome e, opcionalmente, um atributo. Seu nome é um símbolo que representa um tipo de unidade lógica, como uma palavra-chave, um identificador ou um literal. Um padrão é a forma que os lexemas de um *token* podem assumir, sendo geralmente descrito por meio de expressões regulares.

O analisador sintático processa o fluxo de *tokens* para verificar se a sequência respeita as regras gramaticais da linguagem. Sua função é validar a organização das instruções, garantindo que a estrutura do código esteja correta, como o posicionamento de comandos e o correto fechamento de blocos. Ao final, ele organiza essas informações em uma estrutura hierárquica (como uma árvore sintática), assegurando a conformidade do programa antes da próxima etapa da compilação.

Nesse cenário, o presente trabalho consiste no desenvolvimento de uma nova linguagem de programação, chamada de *Project Hail Mary++*, com a abreviação de PHM++, baseada na exploração espacial e no filme de mesmo nome. O foco da linguagem está na abstração de corpos celestes e nos elementos que compõem o Universo. As etapas desse projeto abrangem desde a definição das regras léxicas e sintáticas da linguagem, passando pela especificação de suas expressões regulares e gramática formal, até a construção de autômatos finitos para reconhecimento de padrões léxicos.

2 Desenvolvimento

A criação da linguagem PHM++ foi estruturada a partir da definição de regras baseadas no tema de Astronomia, com o objetivo de utilizar conceitos de corpos celestes e elementos de exploração espacial para personalizar seu funcionamento. O processo incluiu a criação de *tokens* personalizados e suas respectivas expressões regulares, seguidas pelas regras de produção da gramática e, por fim, pela construção de autômatos finitos para o reconhecimento de padrões léxicos.

2.1 Regras da Linguagem

A etapa inicial no desenvolvimento de uma linguagem consiste na definição de sua especificação léxica e sintática. Esse conjunto de regras fundamentais garante a integridade do funcionamento da linguagem e o comportamento previsível do compilador durante a análise do código-fonte. A Tabela 1 apresenta as normas iniciais definidas para a linguagem.

Tabela 1: Especificações da Linguagem PHM++

Normas Iniciais da Linguagem
Inspirada na sintaxe e no funcionamento da linguagem C++.
Os símbolos e <i>tokens</i> têm como base conceitos de exploração espacial.
Utiliza ‘ ; ‘ (ponto e vírgula) para a finalização dos comandos.
Utiliza ‘ { ‘ e ‘ } ‘ (chaves) para a delimitação de blocos de comandos.
Nomes de variáveis devem obrigatoriamente começar com uma letra.
Não é possível utilizar comentários de bloco, apenas comentários de linha, iniciados por ϕ .
O tipo de variável deve ser declarado antes do identificador.
A linguagem é sensível a maiúsculas e minúsculas (case-sensitive).

2.2 Análise Léxica

Para a etapa de análise léxica, a Tabela 2 detalha o conjunto de *tokens* definidos para a linguagem PHM++, relacionando os símbolos de inspiração, as representações adotadas e suas respectivas classificações. Adicionalmente, a Tabela 4 especifica as expressões regulares da linguagem, enquanto a Tabela 5 apresenta as descrições funcionais de cada uma dessas expressões.

Tabela 2: Tokens da Análise Léxica da Linguagem PHM++

Tipo	Símbolo Original	Símbolo PHM++	Descrição	Token Original	Token PHM++
Tipos de Dados	int	electron	Inteiro	T_INT	T_ELECTRON
	float	atom	Ponto Flutuante	T_FLOAT	T_ATOM
	char	neutron	Caractere	T_CHAR	T_NEUTRON
	void	dark_matter	Sem retorno	T_VOID	T_DARK_MATTER
Identificador	id	id	Nomes de Variáveis	T_ID	T_ID
Literal	num	num	Literais Numéricos	T_NUM	T_NUM
Operador de Atribuição	=	=	Atribuição	T_ASSIGN	T_ORBIT_ASSIGN
Operadores Relacionais	<=	<=	Menor/Igual	T_LEQ	T_DWARF
	>=	>=	Maior/Igual	T_GEQ	T_GIANT
	<	<	Menor que	T_LT	T_ASTEROID
	>	>	Maior que	T_GT	T_SUPERNOVA
	!=	!=	Diferença	T_NEQ	T_SINGULARITY
	==	==	Igualdade	T_EQ	T_BINARY
Operadores Lógicos	&&	&&	AND lógico	T_AND	T_CONJUNCTION
			OR lógico	T_OR	T_PARALLEL
	~	~	NOT lógico	T_NOT	T_INVERSION
Operadores Aritméticos	+	+	Soma	T_ADD	T_FUSION
	-	-	Subtração	T_SUB	T_DECAY
	*	*	Multiplicação	T_TIMES	T_COLLISION
	/	/	Divisão	T_DIV	T_SPLIT
	mod	%	Resto	T_MOD	T_REMAINDER
	^	^	Potência	T_POW	T_EXPANSION

Tabela 3: Tokens da Análise Léxica da Linguagem PHM++ (Continuação)

Tipo	Símbolo Original	Símbolo PHM++	Descrição	Token Original	Token PHM++
Símbolos Especiais	.	.	Delimitador	T_DOT	T_POINT
	,	,	Delimitador	T_COMMA	T_SEPARATOR
	:	:	Delimitador	T_COLON	T_MARK
	;	;	Delimitador	T_SCOLON	T_END
	(	(	Delimitador	T_LPAREN	T_ORBIT_OPEN
	)	)	Delimitador	T_RPAREN	T_ORBIT_CLOSE
	{	{	Delimitador	T_LBRACE	T_SYSTEM_OPEN
	}	}	Delimitador	T_RBRACE	T_SYSTEM_CLOSE
	#	#	Especial	T_HASH	T_SIGNAL_MARK
	»	»	Especial	T_SHIFT_RIGHT	T_WARP_OUT
	«	«	Especial	T_SHIFT_LEFT	T_WARP_IN
Palavras Reservadas	//	¢	Comentário	T_DSLASH	T_COMMENT
	if	sun	Controle	T_IF	T_SUN
	else	moon	Controle	T_ELSE	T_MOON
	for	rocket	Loop	T_FOR	T_ROCKET
	while	orbit	Loop	T_WHILE	T_ORBIT
	switch	star	Controle	T_SWITCH	T_STAR
	case	planet	Controle	T_CASE	T_PLANET
	break	escape	Controle	T_BREAK	T_ESCAPE
	return	launch	Retorno	T_RETURN	T_LAUNCH
	void	dark_matter	Tipo*	T_VOID	T_DARK_MATTER
	cin	signal	Entrada	T_CIN	T_SIGNAL
	cout	broadcast	Saída	T_COUT	T_BROADCAST
	define	gravity	Diretiva	T_DEFINE	T_GRAVITY
	include	warp	Diretiva	T_INCLUDE	T_WARP

*O token `T_DARK_MATTER` aparece tanto na categoria *Tipos de Dados* (Tabela 2) quanto em *Palavras Reservadas*. Essa duplicidade é intencional: `dark_matter` funciona simultaneamente como especificador de tipo em declarações de variáveis e funções, e como palavra reservada da linguagem, de forma análoga ao `void` em C++.

Tabela 4: Expressões Regulares da Linguagem

Nome	Expressão Regular
T_NUM	→ [0-9]+(\\.[0-9]+)?
T_ID	→ [a-zA-Z_][a-zA-Z0-9_]*
operando	→ T_ID T_NUM
relacional	→ T_DWARF T_GIANT T_ASTEROID T_SUPERNOVA T_SINGULARITY T_BINARY
aritmetico	→ T_FUSION T_DECAY T_COLLISION T_SPLIT T_REMAINDER T_EXPANSION
logicaBin	→ T_CONJUNCTION T_PARALLEL
contaRelacional	→ T_INVERSION? (T_ID T_NUM) relacional (T_ID T_NUM)
contaArit	→ T_DECAY? operando (aritmetico operando)*
contaLogica	→ contaRelacional (logicaBin contaRelacional)*
condicional	→ contaRelacional (logicaBin contaRelacional)*
tipoVariavel	→ T_ELECTRON T_ATOM T_NEUTRON T_DARK_MATTER

Tabela 5: Descrição das Identificações

Nome	Descrição
T_NUM	Números positivos, inteiros ou reais (sem sinal).
T_ID	Identificador de variáveis.
operando	Elemento utilizado em operações.
relacional	Operadores relacionais.
aritmetico	Operadores aritméticos.
logicaBin	Operadores lógicos binários (AND, OR).
contaRelacional	Expressão relacional, com NOT unário opcional como prefixo.
contaArit	Expressão aritmética, com negação unária opcional como prefixo.
contaLogica	Expressão lógica.
condicional	Estrutura condicional composta.
tipoVariavel	Tipos electron, atom, neutron e dark_matter.

2.3 Análise Sintática

A análise sintática é responsável por verificar se a sequência de *tokens* gerada pela análise léxica está de acordo com a gramática da linguagem PHM++. Nessa etapa, são utilizadas regras de produção para validar a estrutura das instruções, garantindo que o código respeite a organização esperada, como o uso correto de blocos, operadores e comandos.

As produções definidas permitem representar as principais construções da linguagem, como declarações de variáveis, atribuições, estruturas condicionais, laços de repetição e comandos de entrada e saída. A Tabela 6 apresenta as regras de produção desenvolvidas.

Tabela 6: Regras de Produção da Linguagem PHM++

Nome	Regra de Produção
Programa	→ ListaInstrucoes
ListaInstrucoes	→ Instrucao ListaInstrucoes ϵ
Instrucao	→ DecVariavel Atribuicao Condicional Enquanto For Switch Cin Cout Define Include DecFuncao CallFuncao Comentario
DecVariavel	→ tipoVariavel T_ID T_END
Atribuicao	→ T_ID T_ORBIT_ASSIGN Expressao T_END
AtribuicaoFor	→ T_ID T_ORBIT_ASSIGN Expressao
Expressao	→ contaArit T_NUM T_ID
Condicional	→ T_SUN T_ORBIT_OPEN condicional T_ORBIT_CLOSE T_SYSTEM_OPEN ListaInstrucoes T_SYSTEM_CLOSE ParteElse
ParteElse	→ T_MOON T_SYSTEM_OPEN ListaInstrucoes T_SYSTEM_CLOSE ϵ
Enquanto	→ T_ORBIT T_ORBIT_OPEN condicional T_ORBIT_CLOSE T_SYSTEM_OPEN ListaInstrucoes T_SYSTEM_CLOSE
For	→ T_ROCKET T_ORBIT_OPEN AtribuicaoFor T_END condicional T_END AtribuicaoFor T_ORBIT_CLOSE T_SYSTEM_OPEN ListaInstrucoes T_SYSTEM_CLOSE
Switch	→ T_STAR T_ORBIT_OPEN T_ID T_ORBIT_CLOSE T_SYSTEM_OPEN ListaCasos T_SYSTEM_CLOSE
ListaCasos	→ Caso ListaCasos ϵ
Caso	→ T_PLANET T_NUM T_MARK ListaInstrucoes T_ESCAPE T_END
Cin	→ T_SIGNAL T_WARP_OUT T_ID T_END
Cout	→ T_BROADCAST T_WARP_IN T_ID T_END
Define	→ T_GRAVITY T_ID T_NUM T_END
Include	→ T_WARP T_ID T_END

Nome	Regra de Produção
DecFuncao	$\rightarrow$ tipoVariavel T_ID T_ORBIT_OPEN ParamsDec T_ORBIT_CLOSE T_SYSTEM_OPEN ListaInstrucoes RetornoOpc T_SYSTEM_CLOSE
RetornoOpc	$\rightarrow$ T_LAUNCH Expressao T_END $ \epsilon$
CallFuncao	$\rightarrow$ T_ID T_ORBIT_OPEN ParamsChama T_ORBIT_CLOSE T_END
ParamsDec	$\rightarrow$ ListaParamsDec $ \epsilon$
ListaParamsDec	$\rightarrow$ tipoVariavel T_ID MaisParamsDec
MaisParamsDec	$\rightarrow$ T_SEPARATOR tipoVariavel T_ID MaisParamsDec $ \epsilon$
ParamsChama	$\rightarrow$ ListaParamsChama $ \epsilon$
ListaParamsChama	$\rightarrow$ T_ID MaisParamsChama
MaisParamsChama	$\rightarrow$ T_SEPARATOR T_ID MaisParamsChama $ \epsilon$
Comentario	$\rightarrow$ T_COMMENT

Nota sobre Comentario: Em implementações típicas de compiladores, o token `T_COMMENT` é descartado diretamente pelo analisador léxico, antes de ser entregue ao parser. Na gramática da PHM++, optou-se por mantê-lo como uma produção de *Instrucao* para fins de especificação formal completa, indicando que comentários são estruturas sintaticamente válidas em qualquer posição onde uma instrução é esperada. Na implementação prática do analisador léxico, o token pode ser suprimido sem prejuízo à análise sintática.

2.4 Autômatos Finitos

As figuras a seguir representam os autômatos finitos determinísticos (AFD) criados com base nas expressões regulares e regras de produção da linguagem PHM++. Os autômatos de tokens elementares (`T_NUM`, `T_ID` e `T_COMMENT`) são modelados com estados e transições individuais por caractere ou classe de caracteres, refletindo sua estrutura iterativa. Já os autômatos de palavras reservadas e grupos de operadores adotam a representação por cadeia completa em cada aresta, convenção amplamente utilizada na literatura de compiladores para tokens cujo reconhecimento não apresenta ramificações internas relevantes.

- **T_NUM** — Reconhece literais numéricos inteiros e reais positivos. O sinal negativo é tratado como T_DECAY na gramática, eliminando a ambiguidade léxica com o operador de subtração:

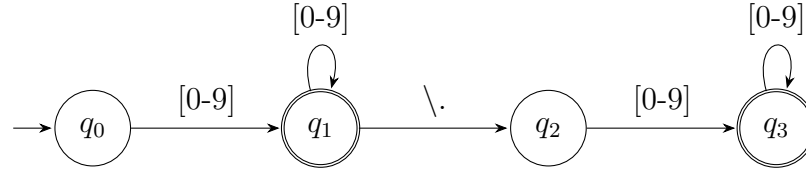


Figura 1: Autômato do token T_NUM.

- **T_ID** — Reconhece identificadores de variáveis, obrigatoriamente iniciados por letra ou sublinhado, seguidos de zero ou mais letras, dígitos ou sublinhados. Espaços não são permitidos:

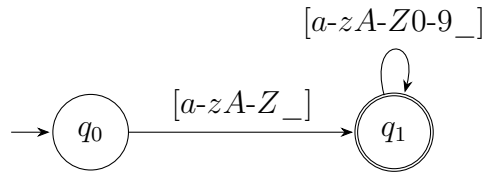


Figura 2: Autômato do token T_ID.

- **T_COMMENT** — Reconhece comentários de linha, iniciados pelo símbolo c e consumindo todos os caracteres subsequentes até encontrar um caractere de nova linha ($\backslash n$), que encerra o comentário:

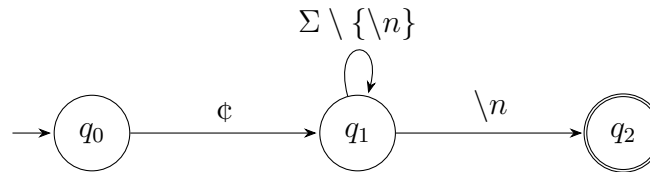


Figura 3: Autômato do token T_COMMENT.

- **Operadores Relacionais** — Reconhece os seis operadores relacionais da linguagem. Os operadores de dois caracteres ($\leq$, $\geq$, $==$, $!=$) consomem um segundo caractere a partir do estado intermediário. Os operadores de um caractere ($<$ e $>$) são aceitos ao encontrar qualquer outro símbolo, sem consumi-lo:

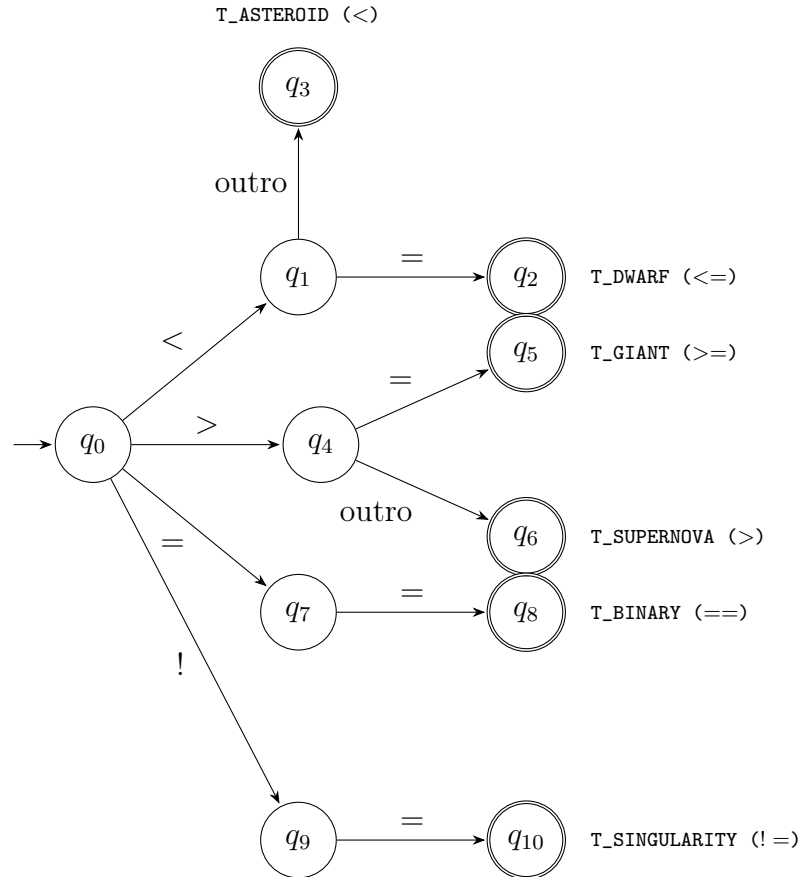


Figura 4: Autômato dos operadores relacionais.

- **Operadores Lógicos** — Reconhece os três operadores lógicos. O operador T_INVERSION ($\sim$) é unário e aceito em um único passo a partir do estado inicial. Os operadores binários T_CONJUNCTION ($\&\&$) e T_PARALLEL ($||$) exigem dois caracteres idênticos consecutivos:

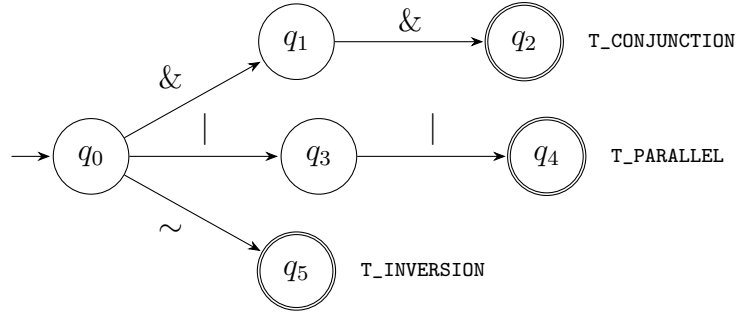


Figura 5: Autômato dos operadores lógicos.

- **Operadores Aritméticos** — Reconhece os seis operadores aritméticos, cada um composto por um único símbolo distinto:

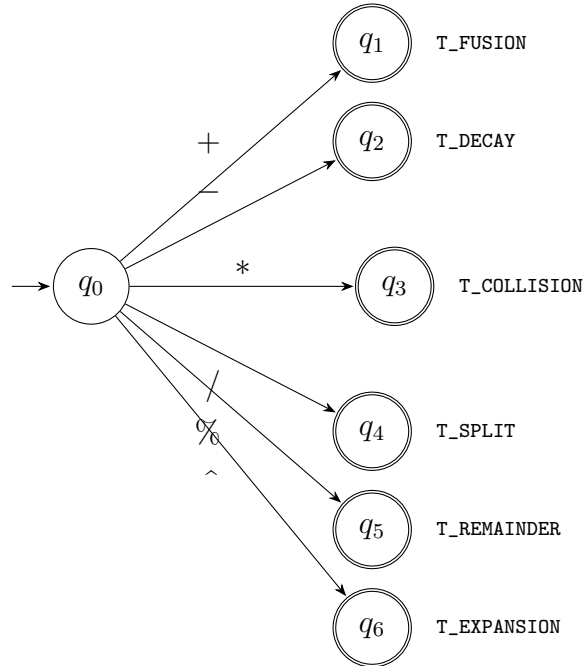


Figura 6: Autômato dos operadores aritméticos.

- **tipoVariavel** — Reconhece as quatro palavras reservadas que representam tipos de dados. Cada aresta carrega a cadeia completa do lexema, pois o reconhecimento de palavras-chave ocorre após o T_ID ter sido lido e comparado à tabela de palavras reservadas:

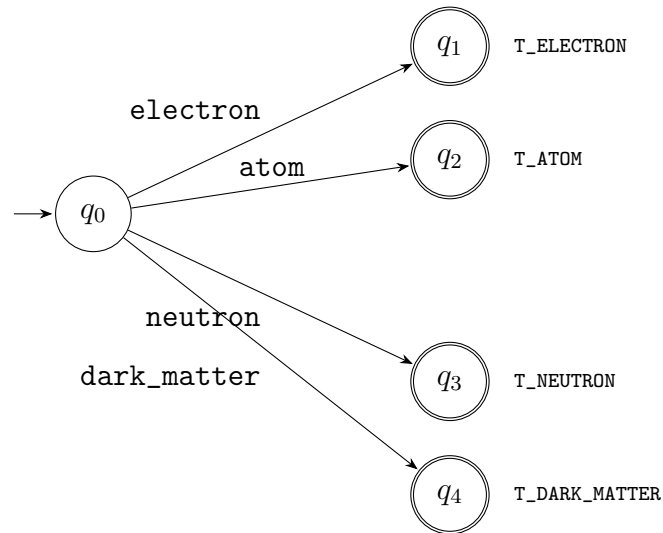


Figura 7: Autômato da expressão `tipoVariavel`.

- **Palavras Reservadas de Controle e Fluxo** — Reconhece as oito palavras reservadas responsáveis pelo controle de fluxo, laços de repetição e retorno de funções na linguagem PHM++:

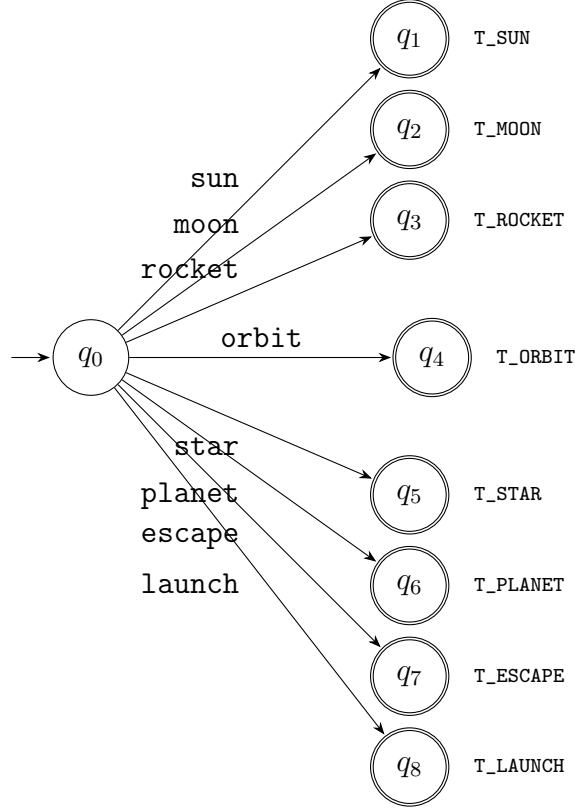


Figura 8: Autômato das palavras reservadas de controle e fluxo.

- **contaRelacional** — Reconhece expressões relacionais com T_INVERSION ($\sim$) como prefixo unário opcional (zero ou um, conforme a expressão regular T_INVERSION?), seguido de um operando, um operador relacional e outro operando. O estado q'_0 garante que no máximo um $\sim$ seja consumido antes do operando:

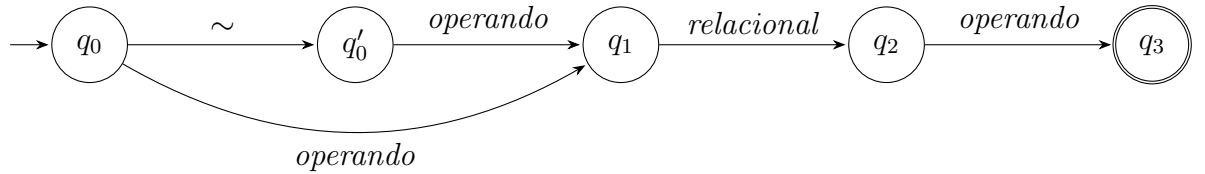


Figura 9: Autômato da expressão contaRelacional.

- **contaArit** — Reconhece expressões aritméticas compostas por operandos separados por operadores aritméticos, com negação unária opcional (T_DECAY) no início:

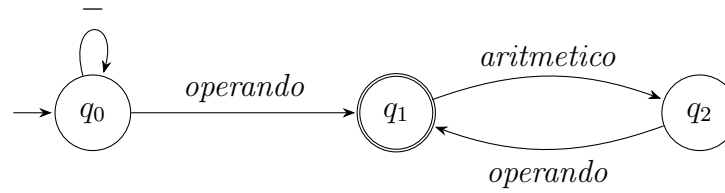


Figura 10: Autômato da expressão **contaArit**.

- **contaLogica** e **condicional** — Reconhecem expressões lógicas formadas por expressões relacionais encadeadas por operadores lógicos binários (T_CONJUNCTION ou T_PARALLEL). A expressão **condicional** possui estrutura idêntica e é utilizada como guarda nas construções **sun**, **orbit** e **rocket**:

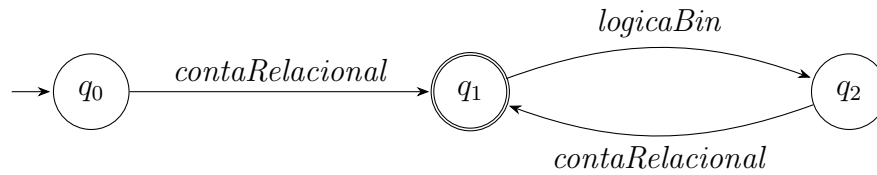


Figura 11: Autômato das expressões **contaLogica** e **condicional**.

3 Exemplo Funcional da Linguagem PHM++

Esta seção apresenta um programa completo escrito em PHM++, acompanhado de anotações que identificam cada construção léxica e sintática utilizada. O programa simula o cálculo da velocidade de escape de um corpo celeste a partir de sua massa e raio, exibindo também a classificação do objeto com base em sua densidade.

3.1 Código-Fonte do Programa

O Código 1 ilustra um programa PHM++ que cobre as principais construções da linguagem: diretivas, declaração e atribuição de variáveis, entrada e saída, estrutura condicional com `sun/moon`, laço `rocket` (equivalente ao `for`), estrutura `star/planet` (equivalente ao `switch/case`) e declaração de função com retorno.

¢

```
1 Programa: Classificador de Corpos Celestes
2 Linguagem: PHM++ baseada no tema de Astronomia
3
4 warp universo ;
5 gravity PI 3 ;
6 gravity LIMITE_DENSO 5 ;
7
8 Funcao que calcula o quadrado de um numero
9 electron quadrado ( electron n ) {
10     electron resultado ;
11     resultado = n * n ;
12     launch resultado ;
13 }
14
15 Funcao principal de classificacao
16 dark_matter classificar ( electron massa , electron raio ) {
17     electron densidade ;
18     electron i ;
19
20     Aproximacao: densidade = massa / raio (simplificado)
21     densidade = massa / raio ;
22
23     Classifica o corpo pelo valor da densidade
24     star ( densidade ) {
25         planet 1 :
26             broadcast << densidade ;
27             escape ;
28         planet 2 :
29             broadcast << massa ;
30             escape ;
31         planet 3 :
```



```

32         broadcast << raio ;
33         escape ;
34     }
35
36     Conta quantas iteracoes sao necessarias ate atingir o
    limite
37     rocket ( i = 1 ; i < LIMITE_DENSO ; i = i + 1 ) {
38         sun ( i == densidade ) {
39             broadcast << i ;
40         }
41     }
42
43     Verifica condicao de corpo compacto ou difuso
44     sun ( densidade >= LIMITE_DENSO ) {
45         broadcast << densidade ;
46     } moon {
47         broadcast << raio ;
48     }
49 }
50
51 Bloco principal do programa
52 electron massa ;
53 electron raio ;
54 electron resultado ;
55
56 signal >> massa ;
57 signal >> raio ;
58
59 Verifica entradas validas antes de classificar
60 sun ( massa > 0 ou raio > 0 ) {
61     classificar ( massa , raio ) ;
62 } moon {
63     broadcast << massa ;
64 }
65
66 Uso do operador logico de inversao (NOT)
67 sun ( ~massa == 0 ) {
68     broadcast << raio ;
69 }
70
71 resultado = quadrado ( raio ) ;
72 broadcast << resultado ;

```

Listing 1: Programa de classificação de corpos celestes em PHM++.

3.2 Mapeamento Léxico e Sintático

A Tabela 7 detalha as principais construções presentes no código, associando cada trecho ao seu padrão léxico ou regra de produção gramatical correspondente.

Tabela 7: Mapeamento léxico e sintático do programa exemplo

Trecho do Código	Token / Regra	Descrição
warp universo ;	<i>Include</i>	Diretiva de inclusão: T_WARP T_ID T_END
gravity PI 3 ;	<i>Define</i>	Definição de constante: T_GRAVITY T_ID T_NUM T_END
electron quadrado (electron n)	<i>DecFuncao</i>	Declaração de função; parâmetro segue ListaParamsDec: tipoVariavel T_ID
resultado = n * n ;	<i>Atribuicao</i>	T_ID T_ORBIT_ASSIGN contaArit T_END; operador T_COLLISION
launch resultado ;	<i>RetornoOpc</i>	Retorno de função: T_LAUNCH Expressao T_END
densidade = massa / raio ;	<i>Atribuicao</i>	Operador aritmético T_SPLIT (divisão)
star (densidade) { ... }	<i>Switch</i>	Seleção múltipla: T_STAR T_ORBIT_OPEN T_ID T_ORBIT_CLOSE ...
planet 1 : ... escape ;	<i>Caso</i>	Caso da seleção: T_PLANET T_NUM T_MARK ... T_ESCAPE T_END
rocket (i = 1 ; i < LIMITE_DENSO ; i = i + 1)	<i>For</i>	Todas as três partes usam AtribuicaoFor; guarda usa T_ASTEROID (<)
i == densidade	<i>condicional</i>	Expressão relacional com operador T_BINARY
sun (densidade >= LIMITE_DENSO)	<i>Condicional</i>	Guarda usa T_GIANT (operador >=); forma T_SUN
moon { ... }	<i>ParteElse</i>	Ramo alternativo: T_MOON T_SYSTEM_OPEN ... T_SYSTEM_CLOSE

Trecho do Código	Token / Regra	Descrição
signal » massa ;	<i>Cin</i>	Leitura de entrada: T_SIGNAL T_WARP_OUT T_ID T_END
broadcast « resultado ;	<i>Cout</i>	Saída de dados: T_BROADCAST T_WARP_IN T_ID T_END
massa > 0 && raio > 0	<i>condicional</i>	Expressão lógica com T_CONJUNCTION; dois contaRelacional encade- ados
~massa == 0	<i>contaRelacional</i>	Operador unário T_INVERSION prefixando a expressão relacio- nal
quadrado (raio) ;	<i>CallFuncao</i>	Chamada de função: T_ID T_ORBIT_OPEN Parametros T_ORBIT_CLOSE T_END
¢ Programa: ...	T_COMMENT	Comentário de linha iniciado pelo símbolo ¢

3.3 Observações sobre o Exemplo

O programa cobre intencionalmente todos os principais *tokens* e regras de produção definidos para a linguagem PHM++. Algumas observações relevantes:

- O token T_DARK_MATTER (equivalente a `void`) é utilizado na declaração de `classificar`, indicando que a função não possui valor de retorno. Por isso, a regra *RetornoOpc* é produzida como ϵ nessa função.
- O laço `rocket` exemplifica a produção *For*, onde a terceira parte (`i = i + 1`) agora é corretamente tratada como *AtribuicaoFor* — e não como *Expressao* — garantindo consistência com a semântica de incremento. A expressão aritmética interna segue *contaArit* com T_FUSION como operador.
- A estrutura `star/planet` utiliza literais numéricos (T_NUM) como rótulos de caso, conforme definido na regra de produção *Caso*. O token T_ESCAPE encerra cada ramo, equivalendo ao `break` do C++.
- O operador T_INVERSION ($\sim$) demonstra o NOT lógico unário na guarda `~massa == 0`. O autômato corrigido de *contaRelacional* utiliza o estado intermediário q'_0 para garantir que no máximo um $\sim$ seja aceito como prefixo, alinhando o autômato à definição T_INVERSION? da expressão regular.
- Os operadores T_WARP_OUT ($\gg$) e T_WARP_IN ($\ll$) são empregados respectivamente nas construções *Cin* e *Cout*, refletindo a inspiração no operador de fluxo do C++.

4 Conclusão

O desenvolvimento da linguagem PHM++ permitiu aplicar, de forma prática, os principais conceitos envolvidos na construção de compiladores, especialmente nas etapas de análise léxica e sintática. A definição de *tokens*, expressões regulares e regras de produção evidenciou a importância de uma especificação formal bem estruturada para garantir a consistência e a previsibilidade do processo de compilação.

Durante a construção da análise léxica, foi possível observar como a modelagem por expressões regulares e autômatos finitos determinísticos viabiliza o reconhecimento eficiente de padrões no código-fonte. A separação clara entre identificadores, palavras reservadas e operadores contribuiu para evitar ambiguidades e facilitar a etapa subsequente.

Na análise sintática, a elaboração da gramática formal destacou a necessidade de organizar corretamente as estruturas da linguagem, assegurando que comandos, blocos e expressões sigam uma hierarquia válida. A introdução de produções específicas para estruturas como condicionais, laços e funções tornou a linguagem suficientemente expressiva, mantendo ao mesmo tempo uma complexidade controlada.

Além disso, a temática baseada em Astronomia trouxe uma camada de abstração diferenciada, demonstrando que é possível personalizar uma linguagem sem comprometer sua coerência estrutural. A adaptação de elementos clássicos de linguagens como C++ para um novo contexto reforçou o entendimento dos mecanismos fundamentais que regem linguagens de programação modernas.

Por fim, o trabalho consolidou o entendimento teórico por meio de uma implementação conceitual completa, abrangendo desde a definição léxica até a representação por autômatos. A revisão crítica da especificação permitiu identificar e corrigir inconsistências relevantes: a separação das produções de parâmetros em *ListaParamsDec* (com tipo, para declarações) e *ListaParamsChama* (sem tipo, para chamadas); a substituição de *Expressao* por *AtribuicaoFor* na terceira parte do laço *For*; a correção do autômato de *contaRelacional* para aceitar no máximo um prefixo `T_INVERSION`, alinhando-o à expressão regular; e a adição de nota formal sobre o tratamento do token `T_COMMENT` na gramática. O exemplo funcional apresentado na Seção 3 demonstrou na prática a coerência entre as especificações formais corrigidas e o comportamento esperado da linguagem. Como continuidade, seria possível evoluir o projeto para incluir análise semântica e geração de código, tornando a linguagem PHM++ ainda mais próxima de um compilador funcional completo.

Referências

- [1] Thatyana de Faria Piola Seraphim. Análise léxica, 2025. Acesso em 05 de maio de 2026.
- [2] Thatyana de Faria Piola Seraphim. Análise léxica: Autômatos finitos, 2025. Acesso em 05 de maio de 2026.
- [3] Thatyana de Faria Piola Seraphim. Introdução à compiladores, 2025. Acesso em 05 de maio de 2026.
- [4] Evan Wallace. Finite state machine designer. <https://madebyevan.com/fsm/>, 2010. Acesso em 05 de maio de 2026.